

PROJECT REPORT
MOVIE LOGGER
DATABASE MANAGEMENT SYSTEMS
FACULTY: Dr.MAHESWARI B

AASHIK MOHAMMED 23BCE2025

SAHAANA M 23BCE2128

MITHUN MADHAN 23BCE0768

MOVIE LOGGER

PROJECT OVERVIEW:

The **Movie Logger Application** is a web-based system designed to help users **record, manage, and review** movies they have watched or plan to watch.

It allows users to add details about movies, update or delete entries, rate them, leave comments, and search through their personal movie collection.

This system demonstrates CRUD (Create, Read, Update, Delete) operations, rating and review features, and efficient data handling using FastAPI and SQLAlchemy.

2. Objectives

- To develop an interactive web application for managing movie logs.
- To implement CRUD operations with an intuitive interface.
- To integrate user-friendly rating and comment features.
- To enable fast searching and filtering of movies.
- To ensure data persistence and easy modification through a database backend.

3. Technologies Used

Component	Technology
Backend Framework	FastAPI
Database ORM	SQLAlchemy
Frontend Template	ReactJS
Styling	CSS
Database	PostgreSQL
Validation	Pydantic Models
Middleware	CORSMiddleware (for cross-origin access)

4.Functionalities

1.CORS Middleware Setup:

```
# CORS setup to allow React frontend
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3001"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Purpose:

Allows the React frontend (running at <http://localhost:3001>) to communicate with the FastAPI backend without CORS (Cross-Origin Resource Sharing) restrictions.

Operations:

- `allow_origins`: Specifies which domains can make requests.
- `allow_methods`: Permits all HTTP methods (GET, POST, PUT, DELETE).
- `allow_headers`: Permits all request headers.

Result: Enables secure cross-origin requests between frontend and backend.

2) Database Table Creation

Purpose:

Creates all database tables defined in `database_models.py` if they don't already exist.

```
# Create database tables
database_models.Base.metadata.create_all(bind=engine)
```

3) Database Session Dependency

```
# Dependency to get DB session
def get_db():
    db = session()
    try:
        yield db
    finally:
        db.close()
```

Purpose:

Provides a database session (db) for each request, ensuring it is properly closed afterward.

How It Works:

- Creates a new session from SQLAlchemy's sessionmaker.
- yield passes the session to FastAPI endpoints.
- After the endpoint finishes, f db.close() is called to release the connection.

4) Database Initialization Function

```
# Initialize DB with sample data if empty
def init_db():
    db = session()
    count = db.query(database_models.Movie).count()
    if count == 0:
        for movie in sample_movies:
            db.add(database_models.Movie(**movie.model_dump()))
        db.commit()
    db.close()

init_db()
```

Purpose:

Initializes the database with sample movie data (only if the table is empty).

Process:

1. Opens a DB session.
2. Checks if the Movie table has records.
3. If empty, loops through sample_movies and adds each one.
4. Commits the changes and closes the session.

Result: Ensures that the database starts with default movie entries (for testing/demo).

5) Get All Movies

```
59 # Get movie by ID
60 @app.get("/movies/{id}")
61 def get_movie(id: int, db: Session = Depends(get_db)):
62     movie = db.query(database_models.Movie).filter(database_models.Movie.id == id).first()
63     if not movie:
64         raise HTTPException(status_code=404, detail="Movie not found")
65     return movie
66
```

Purpose:

Retrieves details of a specific movie using its unique ID.

How It Works:

- Takes the id from the URL path.
- Queries the Movie table for a record matching that id.
- If found → returns movie data.
- If not → raises a 404 Not Found error.

6) Add a New Movie

```
67 # Add a new movie
68 @app.post("/movies/")
69 def add_movie(movie: Movie, db: Session = Depends(get_db)):
70     db_movie = database_models.Movie(**movie.model_dump())
71     db.add(db_movie)
72     db.commit()
73     db.refresh(db_movie)
74     return db_movie
75
```

Purpose:

Creates and adds a new movie record to the database.

Process:

1. Accepts a Movie Pydantic model from the request body.
2. Converts it into a SQLAlchemy model (database_models.Movie).
3. Adds it to the DB session.
4. Commits changes and refreshes the new object to get its ID.
5. Returns the new movie as confirmation.

7) Update Movie Details

```
76 # Update a movie
77 @app.put("/movies/{id}")
78 def update_movie(id: int, movie: Movie, db: Session = Depends(get_db)):
79     db_movie = db.query(database_models.Movie).filter(database_models.Movie.id == id).first()
80     if not db_movie:
81         raise HTTPException(status_code=404, detail="Movie not found")
82     db_movie.name = movie.name
83     db_movie.dateWatched = movie.dateWatched
84     db_movie.rating = movie.rating
85     db_movie.comments = movie.comments
86     db.commit()
87     db.refresh(db_movie)
88     return db_movie
```

Purpose:

Updates the details of an existing movie in the database.

Process:

1. Finds the movie by its id.
2. Updates the record with new data from the Pydantic model.
3. Commits changes to the DB.
4. Returns the updated movie.

8) Delete Movie

```
# Delete a movie
@app.delete("/movies/{id}")
def delete_movie(id: int, db: Session = Depends(get_db)):
    db_movie = db.query(database_models.Movie).filter(database_models.Movie.id == id).first()
    if not db_movie:
        raise HTTPException(status_code=404, detail="Movie not found")
    db.delete(db_movie)
    db.commit()
    return {"detail": "Movie deleted successfully"}
```


Purpose:

Removes a movie record permanently from the database.

Process:

1. Finds the movie by id.
2. Deletes it using SQLAlchemy's delete() method.
3. Commits changes.
4. Returns a success message.

database_models.py**1) Import necessary dependencies**

```
from sqlalchemy import Column, Integer, String, Date
from sqlalchemy.ext.declarative import declarative_base
```

2) Declaring the Base

```
Base = declarative_base()
```

declarative_base():

This creates a base class that all the database models will inherit from. It's what allows SQLAlchemy to understand that a Python class represents a table in the database.

Base acts like a blueprint for building models.

Every model that inherits from Base automatically registers itself with SQLAlchemy's internal mapping system.

3) Creating table:

```
class Movie(Base):  
    __tablename__ = "movies"  
    id = Column(Integer, primary_key=True, index=True)  
    name = Column(String, nullable=False)  
    dateWatched = Column(String, nullable=False)  
    rating = Column(Integer, nullable=False)  
    comments = Column(String, nullable=True)
```

This defines a class Movie that represents a **table named movies** in the database. `__tablename__` specifies the exact name of the table.

Once the table is created the columns and its datatypes are declared.

Models.py:

```
from pydantic import BaseModel, Field
from typing import Optional

class Movie(BaseModel):
    id: int
    name: str = Field(..., example="Inception")
    datewatched: str = Field(..., example="2025-10-01")
    rating: int = Field(..., ge=1, le=5, example=5)
    comments: Optional[str] = Field(None, example="Mind-blowing!")
```

- The necessary dependencies are imported.
- class Movie(BaseModel) creates a Pydantic model which defines how **movie data** should look when being sent or received via an API.
- The datatypes of the column are then declared.

Database.py:

```
1 from sqlalchemy.orm import sessionmaker
2 from sqlalchemy import create_engine
3 db_url = "postgresql://postgres:gatewayheadfc@localhost:5432/movie_db"
4 engine = create_engine(db_url)
5 session = sessionmaker(autocommit=False, autoflush=False, bind=engine)
6
7
```

- **create_engine :**
Used to establish a connection between SQLAlchemy and the PostgreSQL database .
It tells SQLAlchemy *where* the database is and *how* to talk to it.
- **sessionmaker :**
Creates a function that produces new database sessions.
A session is what is used to actually interact with the database — run queries, add data, delete records, etc.
- The connection string tells SQLAlchemy how to connect to the PostgreSQL database.

DATABASE AFTER CRUD OPERATIONS:

Initially the database contains the sample values that was already added at the time of database creation.

	id [PK] integer	name character varying	dateWatched character varying	rating integer	comments character varying
1	1	Inception	2025-10-20	3	ass
2	2	Interstellar	2025-10-21	4	Great visuals.
3	3	The Dark Knight	2025-10-22	5	Best superhero movie oat

CREATE:

Edit made on website:

Add Movie

★ ★ ★ ★ ★

Add

Database after edit:

	id [PK] integer	name character varying	dateWatched character varying	rating integer	comments character varying
1	1	Inception	2025-10-20	3	ass
2	2	Interstellar	2025-10-21	4	Great visuals.
3	3	The Dark Knight	2025-10-22	5	Best superhero movie oat
4	4	Nosferatu	2025-10-28	3	good movie, but a bit outdated as this made in 1922.

READ:

Movies					
Total: 4					
incept					
ID ▲	Name	Date Watched	Rating	Comments	Actions
1	Inception	2025-10-20	★★★★☆	mind bending.	<button>Edit</button> <button>Delete</button>

The desired movies can be read by using the search bar.

UPDATE:

Movie's details can be updated by clicking on the edit option under actions.

Edit Movie

★★★★☆

Update Cancel

Database after update button is clicked:

	id [PK] integer	name character varying	dateWatched character varying	rating integer	comments character varying
1	1	Inception	2025-10-20	3	mind bending.
2	2	Interstellar	2025-10-21	4	Great visuals.
3	3	The Dark Knight	2025-10-22	5	Best superhero movie oat
4	4	Nosferatu	2025-10-28	3	good movie,and it was very scary for a movie made more than 100 years ago

DELETE:

Movies can be deleted using the delete button.

Delete this movie?

OKCancel

Movies

Total: 4

Search by id, name or comr

ID	Name	Date Watched	Rating	Comments	Actions
1	Inception	2025-10-20	★★★☆☆	mind bending.	Edit Delete
2	Interstellar	2025-10-21	★★★★☆	Great visuals.	Edit Delete
3	The Dark Knight	2025-10-22	★★★★★	Best superhero movie oat	Edit Delete
4	Nosferatu	2025-10-28	★★★★☆	good movie,and it was very scary for a movie made more than 100 years ago	Edit Delete

Database after Nosferatu is deleted:

	id [PK] integer	name character varying	dateWatched character varying	rating integer	comments character varying
1	1	Inception	2025-10-20	3	mind bending.
2	2	Interstellar	2025-10-21	4	Great visuals.
3	3	The Dark Knight	2025-10-22	5	Best superhero movie oat

App.js – Frontend Logic

1) Importing Dependencies

Purpose:

Imports essential libraries and components required for UI rendering, state management, HTTP communication, and styling.

Includes:

- React hooks (useState, useEffect, useMemo)
- axios for REST API communication
- Custom stylesheet (App.css)
- Additional UI component (TaglineSection)


```
import React, { useState, useEffect, useMemo } from "react";
import axios from "axios";
import "./App.css";
import TaglineSection from "./TaglineSection";
```

2) API Configuration

Purpose:

Initializes Axios instance to communicate with backend server.

Description:

```
const api = axios.create({
  baseURL: "http://localhost:8081",
});
```

Defines a base URL so all other requests can use relative paths.

```
const api = axios.create({
  |  baseURL: "http://localhost:8081",
  });
```

3) Star Rating Component

Purpose:

Provides a reusable star-based rating UI for movie reviews.

Functionality:

- Displays 5 stars
- Highlights stars based on current value
- Allows interactive rating on click

User Interaction:

Helps visually rate movies during creation or update.

```
// Star Rating Component
const StarRating = ({ rating, setRating }) => {
  return (
    <div className="star-rating">
      {[1, 2, 3, 4, 5].map((i) => (
        <span
          key={i}
          onClick={() => setRating(i)}
          style={{
            cursor: "pointer",
            color: rating >= i ? "gold" : "■ #ccc",
            fontSize: "1.5rem",
            marginRight: "4px",
          }}
        >
          ★
        </span>
      ))}
    </div>
  );
};
```

4) State Management

Purpose:

Maintains and updates application-level UI and data states.

Key States:

- movies — Stores all fetched movie records
- form — Holds input data for add/update operations
- editId — Tracks current movie being edited
- message & error — For UI alerts
- loading — Indicates async processing

- filter — Search field input
- sortField & sortDirection — Sorting control

5) Notifications & Error Handling Logic

Purpose:

Automatically clears success/errors after 5 seconds.

Mechanism:

Uses useEffect timers to reset message and error gracefully.

6) Fetching Movie Data from Backend

Purpose:

Fetches all movies when application loads & on refresh.

Process:

1. Calls /movies/ endpoint
2. Stores response in movies
3. Displays error on failure

```
api.get("/movies/")
```

```
const fetchMovies = async () => {
  setLoading(true);
  try {
    const res = await api.get("/movies/");
    setMovies(res.data);
    setError("");
  } catch (err) {
    setError("Failed to fetch movies");
  }
  setLoading(false);
};
```

7) Sorting & Filtering Logic

Purpose:

Enhances data readability and user experience.

Operations Supported:

- Sort by ID / Name / Rating
- Search movies by id, name, or comments
- Efficient re-render using useMemo

```
const filteredMovies = useMemo(() => {
  let filtered = movies;
  const q = filter.trim().toLowerCase();
  if (q) {
    filtered = movies.filter(
      (m) =>
        String(m.id).includes(q) ||
        m.name?.toLowerCase().includes(q) ||
        m.comments?.toLowerCase().includes(q)
    );
  }
});
```

8) Form Handling & Submission

Purpose:

Add or Update movie records.

Process:

1. Reads input values into form
2. Distinguishes between Create vs Edit using editId
3. Sends data via POST (create) or PUT (update)
4. Refreshes list & resets form after completion

9) Edit Operation

Purpose:

Loads existing movie details into form for modification.

```
const handleEdit = (movie) => {  
  setForm({  
    id: movie.id,  
    name: movie.name,  
    dateWatched: movie.dateWatched,  
    rating: movie.rating,  
    comments: movie.comments,  
  });  
  setEditId(movie.id);  
  setMessage("");  
  setError("");  
};
```

10) Delete Operation

Purpose:

Deletes movie by ID after user confirmation.

Workflow:

- Prompts user
- Calls DELETE /movies/{id}
- Updates list post-deletion

11) Rendering the UI**Purpose:**

Displays movie form, list, search bar, sorting controls, and messages.

Components:

- Header & Refresh button
- Add/Edit movie form
- Search + Table view
- Dynamic star rating display
- Buttons for Edit/Delete
- Tagline section